NAVRACHANA UNIVERSITY

a UGC recognized University

School of Engineering and Technology

Computer Science and Engineering Department

Compiler Endsem

TY B Tech CSE - VI semester

Academic Year: 2025-26

Course name and code

Compiler Design Laboratory(CSE606)

Student name and Enrolment Number

23000317 (Khushi Patel) & 23000337 (Vidhi Shah)

Course In-charge

Prof. Sanjay Kumar Vij

Introduction

The Python-to-C Transpiler is a rule-based source-to-source translation system developed using Python programming language. The main purpose of this project is to convert basic Python code into equivalent C code automatically. Python and C are both widely used programming languages, but they differ significantly in syntax, datatype handling, memory management, and block structure. Python is an interpreted, dynamically typed language that uses indentation for defining blocks, whereas C is a compiled, statically typed language that uses braces and explicit datatype declarations.

This project acts as a mini transpiler that reads Python source code line-by-line, analyzes its syntax using pattern matching and string processing techniques, and generates equivalent C syntax. The system supports fundamental programming constructs such as variable assignments, input/output statements, conditional statements (if, elif, else), loops (for, while), comments, boolean operators, and increment/decrement optimizations.

The project demonstrates important compiler design concepts such as lexical analysis, syntax recognition, symbol table management, stack-based indentation handling, condition conversion using regular expressions, and code generation. The transpiler uses data structures like lists, dictionaries, and stacks to manage source code, variable types, and nested block structures effectively.

This project provides a practical understanding of how source code translation systems and basic compiler mechanisms work internally while simplifying the conversion of beginner-level Python programs into C programs.

Objectives

i) To develop a rule-based Python-to-C transpiler using Python programming language.

ii) To convert basic Python syntax into equivalent C syntax automatically.

iii) To perform line-by-line analysis of Python source code using pattern matching techniques.

iv) To implement datatype detection for integers, floating-point values, and strings.

v) To translate Python conditional statements (if, elif, else) into corresponding C conditional blocks.

vi) To convert Python loop structures (for, while) into equivalent C loop syntax.

vii) To implement boolean operator conversion such as:
And --> &&
Or --> ||
not --> !

viii) To handle Python indentation using stack data structure and generate proper C braces {}.

ix) To create and maintain a symbol table using dictionary data structure for variable type tracking.

x) To convert Python input and print statements into corresponding scanf() and printf() statements in C.

xi) To optimize assignment expressions such as:
- x = x + 1 --> x++
- x += 5 --> x += 5

xii) To understand fundamental compiler design concepts such as:
- lexical analysis
- syntax processing
- source-to-source translation
- code generation

xiii) To generate complete executable C code automatically from Python source code.

Features Implemented in the Project

i. The project supports variable declaration and assignment for integer, float, and string data types by automatically detecting the appropriate data type and generating equivalent C variable declarations.

ii. The converter performs automatic data type detection for integers, floating-point numbers, and strings during the conversion process.

iii. The system supports Python input statements such as int(input()), float(input()), and input(), and converts them into equivalent C statements using scanf() and printf().

iv. The project converts Python print() statements into equivalent C printf() statements and automatically selects the correct format specifier based on the variable type.

v. The converter supports conditional statements including if, elif, and else, and transforms them into equivalent C conditional structures.

vi. The project supports Python while loops and converts them into equivalent C while loops.

vii. The converter supports Python for loops using the range() function, including range(n), range(start, end), range(start, end, step), and negative step values.

viii. The project supports nested looping structures such as nested for loops and nested while loops through indentation-based block management.

ix. The converter supports nested conditional statements and correctly manages multiple levels of nested if-else structures.

x. The project converts Python boolean operators into equivalent C operators by transforming and into &&, or into ||, not into !, True into 1, and False into 0.

xi. The system automatically converts Python comments into equivalent C comments during code generation.

xii. The project performs basic code optimization by converting statements such as x = x + 1 and x += 1 into optimized C statements like x++.

xiii. The converter automatically transforms Python indentation-based blocks into C brace-based blocks by managing opening and closing braces.

xiv. The project uses stack-based indentation tracking to properly handle block creation, nested blocks, and block termination.

xv. The system prevents variable redeclaration by maintaining a variable table and tracking previously declared variables.

xvi. The project detects unsupported Python features such as functions, classes, import statements, and exception handling, and safely terminates execution with an appropriate message.

Algorithm :

START

INPUT number of Python code lines

STORE all Python lines in list pl

INITIALIZE:

cl = empty list for generated C code

vt = empty dictionary for variable types

indent_stack = [0]

stopped = False

FOR each line in pl:

REMOVE empty spaces from line

IF line is empty:

CONTINUE

CALCULATE indentation level

WHILE current indentation is smaller than previous indentation:

ADD "}" to cl

POP indentation stack

IF line is a comment:

CONVERT # comment → // comment

STORE in cl

CONTINUE

IF unsupported syntax detected:

DISPLAY error

```
    stopped = True
    BREAK

IF line is IF statement:
    EXTRACT condition
    CONVERT Python boolean operators to C operators
    GENERATE C if block
    PUSH indentation level

ELSE IF line is ELIF statement:
    EXTRACT condition
    CONVERT condition
    GENERATE else if block
    PUSH indentation level

ELSE IF line is ELSE statement:
    GENERATE else block
    PUSH indentation level

ELSE IF line is WHILE loop:
    EXTRACT condition
    CONVERT condition
    GENERATE C while loop
    PUSH indentation level

ELSE IF line is FOR range() loop:
    EXTRACT loop variable
```

EXTRACT range parameters

DETERMINE start, end, step

DETERMINE loop condition

GENERATE C for loop

PUSH indentation level

ELSE IF line is INPUT statement:

EXTRACT variable name

DETECT datatype

GENERATE variable declaration

GENERATE printf prompt

GENERATE scanf statement

STORE variable type in vt

ELSE IF line is PRINT statement:

EXTRACT printed content

IF string:

GENERATE printf string

ELSE IF variable:

FIND datatype from vt

SELECT proper format specifier

GENERATE printf variable

ELSE:

GENERATE printf expression

ELSE IF line is ASSIGNMENT:

CHECK increment/decrement optimization

IF optimization possible:

GENERATE optimized C code

ELSE:

DETECT datatype

IF variable already declared:

GENERATE reassignment

ELSE:

STORE datatype in vt

GENERATE declaration

END FOR

CLOSE remaining blocks using stack

IF no error:

PRINT final C program

END

Codes :

Project.py

```
n = int(input("Enter number of Python code lines: "))
print("Enter Python code:")
pl = []
for i in range(n):
    pl.append(input())
cl = []
vt = {}
# This function identifies datatype
def get_type(v):
    v = v.strip()
    if v.lstrip('-').isdigit():
        return "int"
    elif v.replace('.', '', 1).lstrip('-').isdigit():
        return "float"
    elif v.startswith('"') or v.startswith("'"):
        return "char*"
    else:
        return "int"
# Converting Python boolean syntax into C boolean syntax
def convert_condition(cond):
    import re #Used for advanced pattern matching and text replacement. and also for regex based transformations.
    # re.sub() Find pattern inside text and replace it.
    cond = re.sub(r'\bnot\s+', '!', cond)
```

```
    cond = re.sub(r'\band\b', '&&', cond)
    cond = re.sub(r'\bor\b', '||', cond)
    cond = re.sub(r'\bTrue\b', '1', cond)
    cond = re.sub(r'\bFalse\b', '0', cond)
    return cond
indent_stack = [0]
stopped = False
# ---------------- MAIN LOOP ----------------
for line in pl:
    if line.strip() == "":
        continue
    indent = len(line) - len(line.lstrip())
    l = line.strip() # contains current Python line without extra spaces. and remove spaces from both start & ends of the line.
    # If current indentation becomes smaller than previous indentation, close block.
    while len(indent_stack) > 1 and indent < indent_stack[-1]:
        cl.append("}")
        indent_stack.pop()
    # convert comments
    if l.startswith("#"):
        comment_text = l[1:].strip()
        cl.append(f'// {comment_text}')
        continue
    # unsupported features
    unsupported = [
        "def ",
```

```
    "class ",
    "return",
    "break",
    "continue",
    "import ",
    "try",
    "except"
]
for kw in unsupported:
    if l.startswith(kw):
        print("\nSorry, this feature is not supported yet.")
        stopped = True
        break
if stopped:
    break




# IF
# contains current Python line without extra spaces.
if l.startswith("if "):
    condition = convert_condition(l[3:].rstrip(":")) # Extract condition and convert it to C syntax
    cl.append(f'if ({condition})' + " {")
    indent_stack.append(indent + 4)
# ELIF
elif l.startswith("elif "):
```

```
        condition = convert_condition(l[5:].rstrip(":"))
        cl.append(f'else if ({condition})' + " {")
        indent_stack.append(indent + 4)
    # ELSE
    elif l.startswith("else"):
        cl.append("else {")
        indent_stack.append(indent + 4)




    # WHILE LOOP
    elif l.startswith("while "):
        condition = convert_condition(l[6:].rstrip(":"))
        cl.append(f'while ({condition})' + " {")
        indent_stack.append(indent + 4)




    # FOR LOOP
    elif l.startswith("for ") and "range" in l:
        parts = l.split() # Split the line into parts based on whitespace. This will help us extract the loop variable and range parameters.
        var = parts[1] # The loop variable is the second part of the line (after 'for').
        range_part = l[l.find("range(")+6 : l.rfind(")")] # +6 is there because moves cursor AFTER "range("   and l.rfind(")" this finds closing bracket
        nums = [x.strip() for x in range_part.split(",")] # if range is "1,5,2" then split will be ["1", "5", "2"] and if range is "5" then nums will be ["5"]
```

```
    if len(nums) == 1:
        start = "0"
        end = nums[0]
        step = "1"
    elif len(nums) == 2:
        start, end = nums
        step = "1"
    else:
        start, end, step = nums
    if step.startswith("-"): # it check if loop is decrementing or incrementing. if step is negative then condition will be var > end otherwise var < end
        condition = f"{var} > {end}" # negative step for decrementing
    else:
        condition = f"{var} < {end}"
    cl.append(
        f'for (int {var} = {start}; {condition}; {var} += {step})' + " {"
    )
    indent_stack.append(indent + 4)



# INPUT
elif "input(" in l and "=" in l:
    var = l.split("=")[0].strip()
    rhs = l.split("=", 1)[1].strip()
    if rhs.startswith("int("):
```

```
            t = "int"
            scan = f'scanf("%d", &{var});'
        elif rhs.startswith("float("):
            t = "float"
            scan = f'scanf("%f", &{var});'
        else:
            t = "char"
            scan = f'scanf("%s", {var});'
        ip = rhs[rhs.find("input(")+6 : rhs.rfind(")")] # extract prompt message
        if t == "char": #string need array in C, so we declare char array of size 100. you can adjust size as needed.
            cl.append(f'char {var}[100];')
        else:
            cl.append(f'{t} {var};')
        if ip.startswith('"') or ip.startswith("'"):
            prompt = ip.replace("'", '"')
            cl.append(f'printf({prompt});')
        cl.append(scan)
        vt[var] = t # store variable type in variable table for later use in print statements




    # PRINT statements
    elif l.startswith("print("):
        inner = l[6 : l.rfind(")")]
```

```
    if inner.startswith('"') or inner.startswith("'"):
        text = inner.replace("'", '"')
        cl.append(f'printf({text});')
    elif inner in vt:
        t = vt[inner]
        if t == "int":
            fmt = "%d"
        elif t == "float":
            fmt = "%f"
        else:
            fmt = "%s"
        cl.append(f'printf("{fmt}\\n", {inner});')
    else:
        cl.append(f'printf("%d\\n", {inner});')


# ASSIGNMENT
elif "=" in l:
    var = l.split("=")[0].strip() # Extract variable name (left side of assignment)
    val = l.split("=", 1)[1].strip() # Extract value/expression (right side of assignment)
    # ---------- INCREMENT / DECREMENT OPTIMIZATION ----------
    # Handles: x = x + 1  →  x++
    #          x = x - 1  →  x--
    #          x += 1     →  x++
    #          x -= 1     →  x--
    #          x = x + n  →  x += n
```

```
#       x = x - n → x -= n
    optimized = False # Flag to track if optimization was applied
    if l.startswith(var + " +=") or l.startswith(var + " -="):
      op = "+=" if "+=" in l else "-=" # Determine if it's an increment or decrement operation
      amount = l.split(op, 1)[1].strip() # Extract the amount being added or subtracted
      if amount == "1":
        cl.append(f'{var}{"++"}' + ';') if op == "+=" else cl.append(f'{var}{"--"}' + ';')
      else:
        cl.append(f'{var} {op} {amount};') # Generate the appropriate C code for the
increment/decrement operation
      optimized = True # conversion already handle
    elif val.startswith(var + " +") or val.startswith(var + " -"):
      if val.startswith(var + " +"):
        amount = val[len(var) + 2:].strip()
        if amount == "1":
          cl.append(f'{var}++;')
        else:
          cl.append(f'{var} += {amount};')
      else:
        amount = val[len(var) + 2:].strip()
        if amount == "1":
          cl.append(f'{var}--;')
        else:
          cl.append(f'{var} -= {amount};')
      optimized = True
    if not optimized:
```

```
            t = get_type(val)
            if var in vt:
                if t == "char*":
                    val = val.replace("'", '"')
                cl.append(f'{var} = {val};')
            else:
                vt[var] = t
                if t == "char*":
                    val = val.replace("'", '"')
                cl.append(f'{t} {var} = {val};')



# Close any remaining open blocks
while len(indent_stack) > 1:
    cl.append("}")
    indent_stack.pop()

# Output generated C code
if not stopped:
    print("\nGenerated C Code:\n")
    print("#include <stdio.h>")
    print()
    print("int main()")
    print("{")
```

```
    for line in cl:
        print("    " + line)
    print()
    print("    return 0;")
    print("}")
```

Cdgui.py

```
from tkinter import Tk, Label, Text, Frame, Scrollbar, Canvas, END, BOTH, RIGHT, Y, LEFT
import subprocess, sys, os

# ── THEME ──────────────────────────────────────────────────────────────────
SKY   = "#01579b"
NAVY  = "#e1f5fe"
WHITE = "#F0F8FF"
LIGHT = "#D6EAF8"
BTN   = "#1565C0"

w = Tk()
w.title("Python → C Converter")
w.state("zoomed")        # fullscreen on Windows/Linux
w.configure(bg=NAVY)
w.resizable(True, True)

# ── TITLE ──────────────────────────────────────────────────────────────────
```

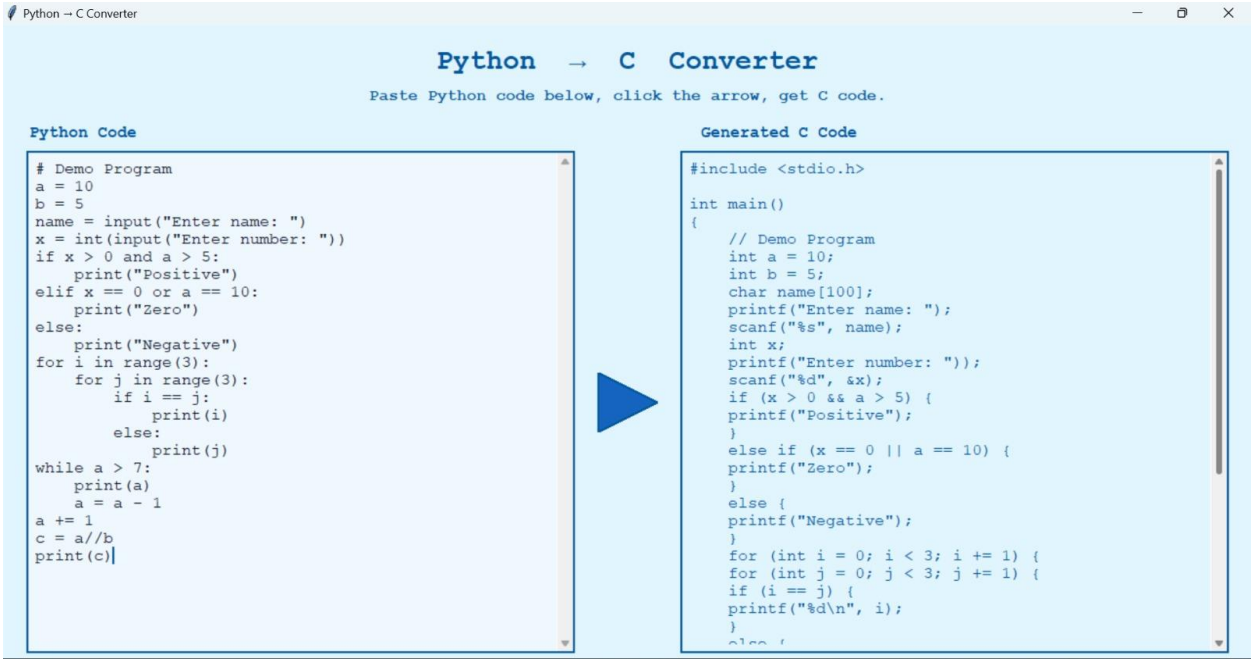
```
Label(w, text="Python → C  Converter",
    bg=NAVY, fg=SKY, font=("Courier", 22, "bold")).grid(
    row=0, column=0, columnspan=3, pady=(18, 4))

Label(w, text="Paste Python code below, click the arrow, get C code.",
    bg=NAVY, fg=SKY, font=("Courier", 12, "bold")).grid(
    row=1, column=0, columnspan=3, pady=(0, 12))

# ── LABELS ──────────────────────────────────────────────────────────────
Label(w, text="Python Code", bg=NAVY, fg=SKY,
    font=("Courier", 13, "bold")).grid(row=2, column=0, padx=24, sticky="w")

Label(w, text="Generated C Code", bg=NAVY, fg=SKY,
    font=("Courier", 13, "bold")).grid(row=2, column=2, padx=24, sticky="w")

# ── INPUT BOX ───────────────────────────────────────────────────────────
in_frame = Frame(w, bg=SKY, bd=2)
in_frame.grid(row=3, column=0, padx=(24, 6), pady=6, sticky="nsew")

in_scroll = Scrollbar(in_frame)
in_scroll.pack(side=RIGHT, fill=Y)

input_box = Text(in_frame, width=40, height=24,
            bg=WHITE, fg="#0d1b2a", insertbackground=SKY,
            font=("Courier", 12), yscrollcommand=in_scroll.set,
            relief="flat", bd=6)
```

```
input_box.pack(side=LEFT, fill=BOTH, expand=True)
in_scroll.config(command=input_box.yview)

# ── TRIANGLE ARROW BUTTON ──────────────────────────────────────

btn_frame = Frame(w, bg=NAVY)
btn_frame.grid(row=3, column=1, padx=8)

def draw_triangle_btn(parent, cmd):
    c = Canvas(parent, width=80, height=80, bg=NAVY, highlightthickness=0)

    # Triangle pointing right (like a play/forward arrow)
    pts = [10, 10, 10, 70, 70, 40]
    tri = c.create_polygon(pts, fill=BTN, outline=SKY, width=2)

    c.bind("<Button-1>", lambda e: cmd())
    c.bind("<Enter>",    lambda e: c.itemconfig(tri, fill="#1976D2"))
    c.bind("<Leave>",    lambda e: c.itemconfig(tri, fill=BTN))
    return c

def convert():
    py_code = input_box.get("1.0", END).strip()
    if not py_code:
        output_box.delete("1.0", END)
        output_box.insert(END, "⚠ Paste some Python code first.")
        return
```

```
    lines = py_code.splitlines()
    n     = len(lines)
    stdin = f"{n}\n" + "\n".join(lines) + "\n"

    script = os.path.join(os.path.dirname(os.path.abspath(__file__)), "project.py")

    try:
        result = subprocess.run(
            [sys.executable, script],
            input=stdin, capture_output=True, text=True, timeout=10
        )
        out = result.stdout.strip()
        if "Generated C Code:" in out:
            out = out[out.index("Generated C Code:") + len("Generated C Code:"):].strip()
    except FileNotFoundError:
        out = "  project.py not found.\n    Keep cdgui.py and project.py in the same folder."
    except Exception as ex:
        out = f"  Error: {ex}"

    output_box.delete("1.0", END)
    output_box.insert(END, out)

tri_btn = draw_triangle_btn(btn_frame, convert)
tri_btn.pack(pady=20)
```

```
# ── OUTPUT BOX ──────────────────────────────────────────────────────────────

out_frame = Frame(w, bg=SKY, bd=2)
out_frame.grid(row=3, column=2, padx=(6, 24), pady=6, sticky="nsew")

out_scroll = Scrollbar(out_frame)
out_scroll.pack(side=RIGHT, fill=Y)

output_box = Text(out_frame, width=40, height=24,
                  bg=NAVY, fg=SKY, insertbackground=SKY,
                  font=("Courier", 12), yscrollcommand=out_scroll.set,
                  relief="flat", bd=6)
output_box.pack(side=LEFT, fill=BOTH, expand=True)
out_scroll.config(command=output_box.yview)

# ── GRID WEIGHTS ────────────────────────────────────────────────────────────
w.grid_columnconfigure(0, weight=1)
w.grid_columnconfigure(1, weight=0)
w.grid_columnconfigure(2, weight=1)
w.grid_rowconfigure(3, weight=1)

w.mainloop()
```

Sample Output :



Features Not Yet Implemented

i. The project does not currently support Python function definitions using the def keyword.

ii. Function calls and parameter passing are not yet converted into equivalent C functions.

iii. Python return statements are not currently supported.

iv. Object-oriented programming concepts such as classes, objects, inheritance, and constructors are not implemented.

v. Python list data structures are not currently supported.

vi. Tuple conversion has not yet been implemented.

vii. Dictionary data structures are not supported in the current version of the project.

viii. The converter currently supports only single-argument print() statements and does not fully handle multiple arguments inside print().

ix. Syntax error detection for issues such as missing colons, invalid indentation, or incorrect loop syntax has not been implemented.

x. Semantic error checking, such as undeclared variable detection and invalid variable usage, is not currently supported.

xi. Array indexing and list indexing operations such as a[0] are not implemented.

xii. Loop control statements such as break and continue are not supported.

xiii. Python exception handling using try and except blocks has not been implemented.

xiv. File handling operations are not supported in the current version.

xv. Advanced mathematical and logical expression handling is still limited.

xvi. Membership operators such as in are not supported.

xvii. Python match-case statements are not converted into C switch-case statements.

xviii. The current implementation is rule-based and line-oriented, and it does not use Abstract Syntax Tree (AST) parsing techniques.

xix. The project currently operates as a console-based application and does not provide a graphical user interface.

Conclusion

This project successfully demonstrates the implementation of a basic Python-to-C code generator. The system is capable of converting fundamental Python constructs such as variable declarations, input/output statements, conditional statements, loops, boolean expressions, and comments into equivalent C code. The project also performs basic optimization and indentation-based block management to generate structured C programs. A graphical user interface was developed using Tkinter to improve user interaction and usability.